



DRONA AVIATION

PlutoBlocks

Guide Book

Every block that programs a Pluto nano drone: what it does, what to try with it, and 18 projects to build.



EDITION 2026.2

DRONES PLUTO X · PLUTO 1.2
DRONAAVIATION.COM

About this guide book

PlutoBlocks is a drag-and-drop block programming application for the Pluto nano drone, developed by Drona Aviation. It needs no prior knowledge of any programming language: you assemble a program by snapping blocks together, and PlutoBlocks turns it into code the drone runs. This handbook documents **all 109 blocks** across all **20 categories**.

Who it is for

Absolute beginners upwards. **No prior programming knowledge is required.** If you can drag a shape with a mouse, you can write a working drone program.

How to read it

IF YOU WANT TO...	GO TO
Understand what PlutoBlocks is and what you need	Chapters 1–2
Learn the four blocks every program needs	Chapter 3
Look up one specific block	Chapters 4–19, or the index on page 29
Build something today	The 18-project workbook, page 25
Fly safely	Page 28

Reading the block tables

Every reference page uses one layout. The block name is on the left and what it does is on the right, usually followed by a **Try** line suggesting something to build with it.

BLOCK	WHAT IT DOES
Set Timer	Starts a countdown for a set number of milliseconds. TRY <i>Change the LED colour every two seconds.</i>

SAFETY FIRST

Propellers are sharp and motors can start without warning. Test every motor and flight program with the **propellers removed** first, fly in a netted or clear open area, and always keep a way to disarm within reach. See page 28.



Written specifically for Pluto, the programmable nano drone.

Designed and built in India by Drona Aviation · © 2026

Contents

PART I · GETTING STARTED

01	What is PlutoBlocks?	4
	Blocks instead of code	
02	The Pluto Platform	5
	Hardware, software and design	
03	Core Blocks	6
	The four blocks every program needs	
·	The 20 Categories at a Glance	7
	Where to find every block	

PART II · PROGRAMMING BASICS

04	LED Control & Console	8
	See and print what your code is doing	
05	Variables	9
	Storing values that change	
06	Math	10
	Arithmetic, rounding, clamping, random	
07	Logic	11
	Decisions, comparisons, AND / OR / NOT	
08	Loops & Timer	12
	Repeating, and acting on time	
09	Create Function	13
	Reusing your own blocks	

PART III · READING THE DRONE

10	Sensor Data	14
	Accelerometer, gyro, compass, barometer	
11	Current State	15
	Live roll, pitch, yaw and height	
12	Flight Status & Flight Mode	16
	Is it safe? Which mode is on?	

13	Remote Control	17
	Reading and overriding the sticks	

PART IV · CONTROLLING THE DRONE

14	Desired State	18
	Commanding target angles	
15	Flight Command	19
	Take off, land, flip, disarm	
16	Motor Drive	20
	Driving M1, M2 and M5–M8	
17	Peripheral Control	21
	Buttons, servos, PWM and ADC	

PART V · SCREEN & SENSING

18	OLED Display	22
	Text, numbers, shapes and frames	
·	OLED Display continued	23
	Shapes, pixels and lines	
19	Ranging Sensors	24
	Distance and obstacle avoidance	

PART VI · BUILD SOMETHING

A	Project Workbook 1–6	25
	Fundamentals you can see	
·	Project Workbook 7–12	26
	Reading the drone	
·	Project Workbook 13–18	27
	Flying and power-ups	
B	Safety & Troubleshooting	28
	Pre-flight checklist	
C	Block Index	29
	All 109 blocks, A–Z	

CHAPTER ONE

What is PlutoBlocks?

PlutoBlocks is a drag-and-drop block programming application for the Pluto nano drone, developed by Drona Aviation. It needs no prior knowledge of any programming language: you assemble a program by snapping blocks together, and PlutoBlocks turns it into code the drone runs.

Blocks instead of typing

Pluto is a development tool as well as a drone. Everything the drone can sense and do is available to you: sensor data, remote-control data, motors, LEDs, screens and more. In C++ you reach those through the API. In PlutoBlocks you reach the same things by dragging a block.

WHAT A BLOCK GIVES YOU

- No syntax to memorise, no semicolons
- Nothing to mistype, blocks only fit where they make sense
- You see the whole program shape at a glance
- Instant feedback through LEDs and the console

WHAT IT STILL TEACHES

- Sequence, selection and repetition
- Variables, types and operators
- Reading sensors and reacting to them
- Real-time and safety-critical thinking

How a PlutoBlocks program runs

Your program runs **alongside** the drone's existing firmware, not instead of it. The flight controller continues to stay level, hold altitude and drive the motors, while your blocks add custom behaviour on top.

DEVELOPER MODE

Your program becomes active only when **Developer Mode** is enabled. Until then the drone flies normally and your blocks sit idle. This is a safety feature: a half-finished program can never take over a flight by accident.

The shape of every program

Each project begins by selecting a receiver mode, then uses three lifecycle blocks, one that runs once at the start, one that repeats, and one that runs at the end. Chapter 3 covers these in detail.

STAGE	WHAT BELONGS THERE
Select Receiver Mode	Choose WiFi or external RC. Always first.
on start	Set up once when Developer Mode turns on, variables, screen, sensors, starting LED state.
pluto loop	Your main program. Runs over and over while Developer Mode is on.
on stop	Runs once when Developer Mode turns off. Clean up safely, LEDs off, motors stopped.

CHAPTER TWO

The Pluto Platform

The platform has three parts. **Hardware**, **Software** and **Design**. All three matter for safe flight, and all three can be customised for your own projects.

COMPONENT	ROLE
Hardware	The electronics and mechanical parts that make and control the drone.
Software	Flight logic, communication and the programming interfaces. Magis firmware, PlutoBlocks, the API.
Design	The frame and structure, customisable for new ideas or extra components.

Primus · the flight controller

DRONE	FLIGHT CONTROLLER	VERSION
Pluto X	Primus X2 v1	Current
Pluto 1.2	Primus V5	Current
Pluto X	Primus X	Legacy
Pluto 1.2	Primus V4	Legacy

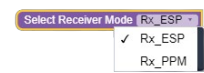
FEATURE	DETAIL	NOTES
Microcontroller	STM32F303CB	32-bit ARM Cortex-M4 · 72 MHz · 128 KB flash
Sensors	ICM-20948	9-axis IMU, accelerometer, gyroscope, magnetometer
	ICP-10111	Barometer, pressure and temperature
	INA219	Voltage and current monitoring
Power	2.8 – 5.5 V in	3.3 V logic · 1S battery charger (0.9–1 A)
Motor driver	4 × MOSFET (6 A)	Plus 2 × bi-directional drivers on X2 only
Wireless	WiFi	Access Point or Station mode
Onboard ports	SPI · I ² C · PPM	5 × ADC · 3 × PWM · 6 × GPIO · camera
Physical	48 × 48 mm · 10.4 g	3.6 mm mounting holes
Programming	AeroFlash OTA	Or USB DFU mode
Unibus	20-pin header	For add-on modules. Pluto X only

CHAPTER THREE

Core Blocks

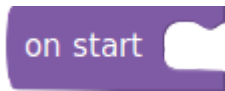
Four blocks form the skeleton of every PlutoBlocks program. They select the receiver, run your setup once, repeat your main logic, and shut things down safely. Your program runs alongside the drone's own firmware, and only while Developer Mode is enabled.

Select Receiver Mode



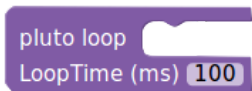
Function	Configures the drone to use either WiFi (Rx_ESP) or External RC (Rx_PPM) receivers.
Setup	Choose Rx_ESP for smartphone control or Rx_PPM for Flysky/standard remote controllers.

on start



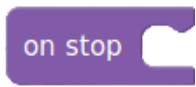
Function	Initializes parameters and runs code once when Developer Mode is turned ON.
Usage	Set initial variable values, calibrate sensors, or establish baseline parameters before flight.

pluto loop



Function	The primary execution loop that runs continuously with a defined interval.
Usage	Place your main flight logic here. Default interval is 100ms for stable performance.

on stop



Function	Executes cleanup and safety routines once when Developer Mode is turned OFF.
Usage	Ensure motors are disarmed, LEDs are reset, or data logs are closed safely.

Pluto Program Execution

On Start	Runs only once when Developer Mode is turned ON.
Pluto Loop	Runs continuously as long as Developer Mode remains ON.
On Stop	Runs only once when Developer Mode is turned OFF, at the end of the execution.

The 20 Categories at a Glance

PlutoBlocks has 109 blocks in 20 categories. This is the whole palette on one page — find the category, then turn to its page.

CATEGORY	WHAT IT IS FOR	BLOCKS	PAGE
1. Core Blocks	The four blocks that give every program its shape: pick a receiver, set up, loop, stop.	4	6
2. LED Control	Three onboard LEDs, red, green and blue, for instant visual feedback.	3	8
3. Create Function	Group blocks under a name and reuse them, or drop in your own C++.	3	13
4. Timer	Millisecond countdowns, so things happen on the clock rather than every pass.	3	12
5. Peripheral Control	Buttons, switches, servos and analog sensors wired to the GPIO, PWM and ADC pins.	5	21
6. Flight Command	High-level flight: take off, land, arm, flip and the disarm kill-switch.	5	19
7. Motor Drive	Direct speed control of the individual motors, for bench tests and ground robots.	6	20
8. Remote Control	Read the pilot sticks, or write stick values from your own program.	8	17
9. Sensor Data	Raw readings from the IMU, compass, barometer and battery monitor.	5	14
10. Current State	The live attitude and height the flight controller has worked out.	5	15
11. Desired State	Ask for a target roll, pitch or yaw and let the firmware fly it.	3	18
12. Flight Mode	Check which flight mode is currently active.	2	16
13. Flight Status	The safety flags: ready to arm, armed, low battery, signal lost, crashed.	6	16
14. Console	Print messages and live values to the Pluto Monitor while the program runs.	2	8
15. Logic	Conditions and decisions: if-then, comparisons, AND, OR and NOT.	7	11
16. Loops	Repeat work without duplicating blocks, and break out when you need to.	4	12
17. Math	Arithmetic, rounding, limits, constants, trigonometry and random numbers.	12	10
18. Variables	Named stores for values that change while the program runs.	3	9
19. OLED Display	Draw text, numbers, shapes and faces on the drone's OLED screen.	16	22
20. Ranging Sensors	Laser rangefinders that measure distance and can avoid obstacles on their own.	7	24

NOT SURE WHERE TO START?

Work through Part II in order. LEDs, variables, math, logic, loops. Every one of those you can see or print without the drone leaving the ground, and they are the foundation for everything in Parts III–V.

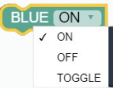
CHAPTER FOUR

LED Control & Console

The two ways to see what your program is doing: light on the drone, and text on your screen.

LED Control

Every Pluto flight controller (Primus X, V4, X2 and V5) carries three programmable LEDs: red, green and blue. They are the quickest way to see what your program is doing without looking at a screen.

BLOCK	WHAT IT DOES
	Red LED Sets the Red LED to ON, OFF or TOGGLE. <i>TRY Real-time emergency warning system that activates on sensor fault, low battery threshold, or collision detection for immediate visual feedback.</i>
	Green LED Sets the Green LED to ON, OFF or TOGGLE. <i>TRY Flight-ready status indicator that signals drone armed state, autonomous mission initialization, or successful system calibration completion.</i>
	Blue LED Sets the Blue LED to ON, OFF or TOGGLE. <i>TRY Network connectivity or telemetry link status display, sensor calibration phase indicator, or authentication state confirmation for secure operations.</i>

Console

Print text and values to the Pluto Monitor on your computer while the program runs. This is how you find out what your program is really doing.

BLOCK	WHAT IT DOES
	Print Message Sends text to the Pluto Monitor app. <i>TRY Real-time mission debugging, autonomous logic verification, or flight event logging for post-mission analysis.</i>
	Print Text + Value Shows a label with a live sensor number. <i>TRY Live sensor telemetry monitoring, variable state tracking during flight, or sensor data logging for performance optimization.</i>

ON, OFF AND TOGGLE
There is no blink state. You make an LED blink by putting *TOGGLE* inside the *pluto loop*, so each pass flips it. Add a Timer (Chapter 8) to control the speed.

A DEBUGGING HABIT
When a program misbehaves, print the value you think is wrong. Usually the number on screen is not the number you expected, and that shows you which part to fix.

CHAPTER FIVE

Variables

A variable is a named store that holds a value while your program runs. Put a value in, read it later, change it when you need to.

A variable is a named store that holds a value while your program runs. Create it once, then read and change it as the program goes.

BLOCK	WHAT IT DOES
Create Integer	Create Integer Declares and initializes an integer variable (whole numbers) in memory. <i>TRY Counting loop iterations, counting execution attempts, tracking program states, or keeping track of flip counts.</i>
Create Float	Create Float Declares and initializes a float variable (decimal numbers) to store fractional values. <i>TRY Storing sensor readings with decimals, tracking exact roll/pitch angles, PID gain parameters, or distance calculations.</i>
Create Boolean	Create Boolean Declares and initializes a boolean variable (true or false) to represent binary state flags. <i>TRY Creating a toggle switch for LEDs, marking a mission task as complete, safety guards, or conditional arming gates.</i>

Choosing a type

TYPE	HOLDS	USE IT FOR
Integer	Whole numbers	Counters, RC values (1000–2000), motor PWM, heights in cm
Float	Numbers with decimals	Averages, sensor maths, anything divided
Boolean	true or false	Flags — "has taken off", "warning shown", "button was pressed"

Set vs. Change

BLOCK	EFFECT ON A VARIABLE CURRENTLY HOLDING 10
Set Variable to 1	It becomes 1 , the old value is replaced.
Change Variable by 1	It becomes 11 , the value is added to.

THE CLASSIC MISTAKE

Creating a variable inside *pluto loop* resets it on every pass of the loop, so it never counts up. Create and initialise variables in *on start*; only change them in the loop.

NAME VARIABLES CLEARLY

`batteryPercent` says what the value holds. `x` does not. Clear names make a program far easier to come back to.

CHAPTER SIX

Math

Arithmetic, rounding, limits and randomness: the blocks that turn raw numbers into decisions.

Work with numbers: arithmetic, rounding, keeping values inside safe limits, trigonometry, constants and randomness.

BLOCK	WHAT IT DOES
	Number A fixed number you can drop into any value slot.
	Arithmetic Performs basic arithmetic operations.
	Single Operation Performs a single mathematical operation, such as square root.
	Trigonometry Performs trigonometric operations.
	Constant Supplies a mathematical constant such as π .
	Number Property Checks if a number is even, odd, prime, whole, positive, negative, or divisible by a certain number.
	Round Rounds a number to the nearest integer. TRY Tidy a calculated average before printing it, or turn a decimal sensor reading into a whole number for the OLED.
	Modulo Returns the remainder of a division.
	Clamp (Limit) Keeps a number inside a range. TRY Motor control safety bounds enforcement, sensor value normalization, or command output range validation.
	Random Integer Picks a whole number randomly.
	Random Fraction Picks a random number between 0 and 1.
	Atan2 Takes two numbers, X and Y , and returns the angle of the point (X, Y) measured from the x-axis, in degrees. Unlike a plain arctangent it uses the sign of both inputs, so it gives the correct angle in all four quadrants. TRY Work out the heading to a target from its X and Y offsets.

TRY · SEVERAL OF THE ABOVE Sensor data processing, control value calculation, or any other mathematical calculation.

TRY · SEVERAL OF THE ABOVE Randomized flight patterns for coverage missions, stochastic behavior simulation, or non-deterministic autonomous routines.

CLAMP IS A SAFETY BLOCK

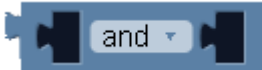
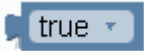
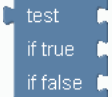
Motor and RC values must stay inside 1000–2000. Clamping a calculated value before you send it means a maths slip can never command something the drone cannot do.

CHAPTER SEVEN

Logic

Decisions. Compare values, combine several checks, and let the drone choose what to do.

Compare values, combine several checks, and let the drone decide what to do next.

BLOCK	WHAT IT DOES
	If This, Then That Runs the blocks inside it only when the condition is true. Click the gear icon on the block to add else if branches for further conditions and a final else branch that runs when none of them match. TRY Conditional flight path planning, sensor-based autonomous decision making, or altitude-dependent thrust adjustments.
	Both True? (AND) Joins two conditions into one. The dropdown switches the block between and (both conditions must be true) and or (either one is enough), so a single block covers both cases. TRY Complex multi-factor decision logic, failsafe condition combinations, or cooperative sensor-based automation triggers.
	Compare Compares two values and reports true or false. The dropdown selects the test: = equal, ≠ not equal, < less than, ≤ less than or equal, > greater than, ≥ greater than or equal. TRY Sensor value thresholding, state comparison, or event-triggering based on value equality.
	Not Inverts a condition. TRY State inversion, condition negation, or toggling between two states.
	Boolean Represents a true or false value. TRY Flagging events, tracking states, or representing binary conditions.
	Null Represents an empty or null value. TRY Initializing variables, representing empty values, or checking for the absence of a value.
	Ternary A compact if-then-else block. TRY Conditional value assignment, state-based value selection, or compact conditional logic.

Hidden options on these blocks

Three of these blocks do more than their picture suggests. The dropdown and gear controls change what the block does.

BLOCK	HOW TO CHANGE IT	OPTIONS
If This, Then That	Gear icon on the block	Add else if branches for more conditions, and one else branch that runs when none match.
Compare	Dropdown in the middle	= · ≠ · < · ≤ · > · ≥
Both True? (AND)	Dropdown in the middle	and both conditions must be true · or either one is enough

Building a condition

YOU WANT	BLOCKS TO COMBINE
Green LED on when the counter passes 20	<i>If-Then</i> + <i>Compare</i> (counter > 20)
...but only if the battery is healthy	Add <i>AND</i> + <i>NOT</i> (Is Low Battery)
Warn on low battery or lost signal	<i>OR</i> across two Flight Status blocks

TEST CONDITIONS ON THE GROUND

Wire any new condition to an LED first, then connect it to something that moves once the light behaves correctly.

CHAPTER EIGHT

Loops & Timer

Repeat without copying blocks, and act on the clock rather than on every pass of the loop.

Loops

Run the same instructions many times without copying blocks, and stop early when a condition is met.

BLOCK	WHAT IT DOES
	For Loop (Counter) Repeats an action for a specific count.
	Repeat While Repeats code as long as a condition is met. TRY Persistent condition monitoring, altitude-dependent motor control, or sensor-driven continuous operation loops.
	Repeat Repeats an action for a specific number of times.
	Break/Continue Exits or skips an iteration of a loop. TRY Emergency loop termination, skipping an iteration based on a condition, or creating complex loop control flow.

TRY · SEVERAL OF THE ABOVE Repetitive status indication patterns, sequential motor testing routines, or counted autonomous flight sequences.

Timer

Timers let you act after a set number of milliseconds instead of on every pass of the loop, which is how you control the speed of anything that repeats.

BLOCK	WHAT IT DOES
	Set Timer Sets up a countdown clock in milliseconds (1000 ms = 1 second) and chooses if it should start counting down again automatically when it finishes. TRY Keep the drone hovering in one spot for exactly 3 seconds, turn a warning light on for 1 second, or perform a safety check every 5 seconds.
	Reset Timer Restarts your countdown clock back from the beginning, just like pressing the reset button on a stopwatch. TRY Restarting a safety countdown when a remote command is received, or starting a fresh hover period after completing a flight movement.
	Check Timer Checks if your countdown clock has finished. It says "Yes" (true) if the time is up, and "No" (false) if it is still counting. TRY Automatically landing the drone after a 5-second countdown finishes, changing direction when a phase timer runs out, or flashing a light every time a cycle is complete.

NEVER BLOCK THE LOOP

pluto loop shares time with the flight controller. A long *Repeat* that runs for seconds will stall flight control. Use a *Timer* to spread work across loop passes instead of holding the loop up.

TIMER, NOT COUNTING LOOPS

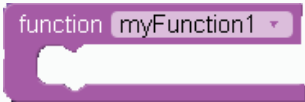
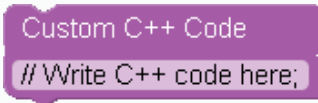
Counting loop passes to measure time is unreliable, the loop rate varies. *Set Timer* and *Check Timer* give you real milliseconds.

CHAPTER NINE

Create Function

Group blocks under a name and reuse them, instead of repeating the same group in several places.

Group a set of blocks under a name and call it whenever you need it, instead of repeating the same group in several places. You can also embed your own C++ here.

BLOCK	WHAT IT DOES
	Define Function Code Groups several blocks into a single named action. <i>TRY Modular safety validation routine that encapsulates battery health checks, RC signal verification, and sensor calibration into reusable custom logic.</i>
	Call Function Code Executes the logic stored within a defined function. <i>TRY Execute recurring drone operations like safety pre-flight checks, state monitoring routines, or complex maneuvers repeatedly throughout autonomous missions.</i>
	Custom C++ Code Embed raw C++ code for advanced low-level tasks. <i>TRY Advanced drone algorithms, integration of existing flight libraries, real-time signal processing, or high-performance computational tasks beyond standard blocks.</i>

When to make a function

SIGN	WHAT TO DO
The same group of blocks appears more than twice	Name it and call it instead.
A chunk of the loop needs a comment to explain it	The comment is the function name you need.
You want the same warning from several conditions	One Warning() function, called from each.

WORKED EXAMPLE

Build a **Warning()** function that blinks the red LED and prints a message. Then call it from *Is Low Battery* and again from *Is Crashed*. Change the warning once, and both places update.

CUSTOM C++

PlutoBlocks also lets you embed your own C++ inside a function block, a bridge to the full API when you outgrow the palette. See the Pluto IDE API Handbook for what is available there.

CHAPTER TEN

Sensor Data

Raw readings from the onboard hardware: motion, magnetic field, air pressure and battery health.

Raw readings straight from the hardware on board: motion, rotation, magnetic field, air pressure and the battery monitor.

BLOCK	WHAT IT DOES
Acceleration (cm/s ²) X ▾	Accelerometer Linear acceleration in cm/s ² on the axis chosen in the dropdown (X, Y or Z). <i>TRY Impact-triggered safety responses, gesture-based flight control through throw-to-launch detection, or flight intensity monitoring for maneuver analysis.</i>
Gyroscope (deciDegree/s) X ▾	Gyroscope Rotation rate in deci-degrees per second on the axis chosen in the dropdown. <i>TRY Spin rate-triggered actions, rotation speed monitoring for acrobatic maneuvers, or angular velocity-based flight behavior modification.</i>
Magnetometer (microTesla) X ▾	Magnetometer Magnetic field strength in microTesla on the axis chosen in the dropdown. For a heading, use <i>Current Yaw</i> instead. <i>TRY Print all three axes and rotate the drone by hand to see the field change.</i>
Barometer pressure (100* millibar) ▾	Barometer Returns current atmospheric pressure (in 100* millibar) or temperature (in 100* °C) based on the selection. <i>TRY Launch-on-blow detection (triggering takeoff by detecting sudden pressure changes when blowing on the drone), altitude estimation, or temperature logging.</i>
Battery Status Voltage (mV) ▾	Battery Status Battery readings from the onboard monitor, chosen in the dropdown: voltage in mV, current, and mAh consumed or remaining. <i>TRY Warn with the red LED when the voltage falls below a level you choose.</i>

RAW VS. WORKED OUT

Sensor Data gives you what the chips measure. Current State (Chapter 11) gives you what the firmware has *worked out* from them, angles and height. For "how is the drone oriented", use Current State; for "what is the hardware seeing", use Sensor Data.

BUILD A DASHBOARD

Print all four on a loop with a Timer at 500 ms, then tilt and move the drone by hand and watch which numbers respond to what.

CHAPTER ELEVEN

Current State

The drone's live orientation and altitude, already filtered and fused by the flight controller.

The drone's live orientation and height, already filtered and fused by the flight controller. Use these rather than raw sensor data when you want to know how the drone is sitting.

BLOCK	WHAT IT DOES
 Current Roll (degrees)	Current Roll Real-time roll angle in degrees. <i>TRY Precision attitude maintenance for level flight, acrobatic maneuver verification, or feedback-based stability control algorithms.</i>
 Current Pitch (degrees)	Current Pitch Real-time pitch angle in degrees. <i>TRY Maintain forward flight angle or detect stall conditions.</i>
 Current Yaw (degrees)	Current Yaw Real-time yaw angle in degrees. <i>TRY Heading lock implementation or rotation tracking.</i>
 Current Height (cm)	Current Height Altitude relative to takeoff (cm). <i>TRY Altitude ceiling enforcement, terrain-aware autonomous flight, or altitude-triggered mission phase transitions.</i>
 Get App Heading	Get App Heading Returns the smartphone's heading angle (0 to 360 degrees) from the Pluto Controller app. <i>TRY Phone-guided heading lock (aligning the drone's yaw orientation with the phone's heading), phone-controlled yaw steering, or interactive remote flight modes.</i>

A SPIRIT LEVEL IN FIVE BLOCKS
Read Current Roll, then light the green LED when it sits near zero and the red LED when it does not. That gives you a working levelling tool and confirms you can read attitude reliably.

HEIGHT NEEDS A REFERENCE
Height is measured from where the drone started. Set it down, power up, and zero is that spot.

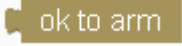
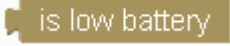
CHAPTER TWELVE

Flight Status & Flight Mode

Is it safe to fly, and which mode is active? These blocks underpin any safety system you build.

Flight Status

The drone's safety flags. These blocks are the foundation of any safety system you build, and of every automatic take-off.

BLOCK	WHAT IT DOES
 Ok to arm	Ok to Arm True when pre-flight checks pass and the drone is safe to arm. <i>TRY Gate every automatic take-off behind this block.</i>
 Is armed	Is Armed True when the motors are live. <i>TRY Light the blue LED whenever the drone is armed.</i>
 Is Signal lost	Is Signal Lost True when the drone stops receiving commands from the phone or transmitter, so the failsafe takes over. <i>TRY Auto-land or level off the moment the pilot link drops.</i>
 Is inflight low battery	Is Inflight Low Battery Checks whether the battery voltage falls below the safe threshold while flying. <i>TRY Trigger automatic emergency landing or flash the Red LED to alert the pilot of low battery during flight.</i>
 Is low battery	Is Low Battery Checks if the drone battery is low under no load (before takeoff). <i>TRY Block takeoff if battery is low, and trigger battery charging/maintenance alerts.</i>
 Is crashed	Is Crashed True when a crash has been detected. <i>TRY Disarm immediately and blink the red LED.</i>

Flight Mode

Detect which flight mode is currently active, so your program can behave differently in each one.

BLOCK	WHAT IT DOES
 Is headfree mode	Headfree Mode Checks if drone is in direction-relative mode. <i>TRY Mode-specific LED indicators, flight mode confirmation display, or mode-dependent behavior switching for dynamic flight control.</i>
 Is throttle mode	Throttle Mode Checks if the drone is in throttle (manual power) mode. <i>TRY Visual feedback for throttle-mode status, or mode transition alerts.</i>

A complete status indicator

CONDITION	SHOW
<i>Ok to Arm is true</i>	Green LED, ready to fly
<i>Is Armed is true</i>	Blue LED, motors are live
<i>Is Low Battery, Is Signal Lost or Is Crashed</i>	Red LED, stop and check

CHECK BEFORE YOU COMMAND

Always confirm *Ok to Arm* before triggering a take-off from blocks. If the drone is not level, still calibrating, or low on battery, arming should not happen, and this is the block that tells you.

CHAPTER THIRTEEN

Remote Control

Read what the pilot is doing on the sticks, and, when you are ready, write stick values from your program.

Read what the pilot is doing on the sticks, and, when you are ready, write stick values from your program to fly it yourself.

BLOCK	WHAT IT DOES
	Read RC Roll Reads current Roll stick input (1000-2000). <i>TRY Detect banking commands for custom gestures.</i>
	Read RC Pitch Reads current Pitch stick input (1000-2000). <i>TRY Monitor forward/backward commands.</i>
	Read RC Throttle Reads current Throttle stick input (1000-2000). <i>TRY Trigger altitude-based events.</i>
	Read RC Yaw Reads current Yaw stick input (1000-2000). <i>TRY Detect rotation commands.</i>
	Set RC Roll Overrides Roll channel with custom value. <i>TRY Automated banking maneuvers.</i>
	Set RC Pitch Overrides Pitch channel with custom value. <i>TRY Autonomous forward flight control.</i>
	Set RC Throttle Overrides Throttle channel with custom value. <i>TRY Auto-hover or altitude hold logic.</i>
	Set RC Yaw Overrides Yaw channel with custom value. <i>TRY Automated rotation or heading lock.</i>

STICK VALUES

Every channel reads **1000–2000**. 1000 is fully back or left, **1500 is centre**, 2000 is fully forward or right.

SETTING CHANNELS TAKES CONTROL FROM THE PILOT

Read blocks are always safe. Write blocks override the human at the controls, test them with propellers off, and always return channels to 1500 when your manoeuvre ends.

CHAPTER FOURTEEN

Desired State

Ask for a target angle and let the flight controller work out how to reach it and stay stable.
Ask the flight controller for a target angle. It works out how to reach it and keeps the drone stable on the way.

BLOCK	WHAT IT DOES
desired roll (degrees)	Desired Roll Sets target roll angle. <i>TRY Coordinated banking maneuvers.</i>
desired pitch (degrees) 0°	Desired Pitch Sets target pitch angle. <i>TRY Controlled forward/backward flight speed.</i>
desired yaw (degrees) 0°	Desired Yaw Sets target yaw rate/angle. <i>TRY Smooth panning for video shots.</i>

Desired State vs. Motor Drive

APPROACH	WHAT HAPPENS
Desired State , you ask for a 10° lean	The firmware keeps the drone stable and leans it. Safe, self-correcting. <i>Prefer this.</i>
Motor Drive , you set motor power directly	You bypass stabilisation entirely. Nothing catches your mistake. Bench use only.

SMALL AND BRIEF

Start with a small angle for a short time, then return to level. A large desired angle held for seconds will send the drone across the room.

ALWAYS COME BACK TO LEVEL

End every guided manoeuvre by setting the desired angle back to 0. If you do not, the drone keeps leaning.

CHAPTER FIFTEEN

Flight Command

The high-level commands: take off, land, flip and disarm. Pair every one of these with a safety check.

The high-level flight commands. Each one hands a whole manoeuvre to the flight controller, so pair every one of them with a safety check.

BLOCK	WHAT IT DOES
take off 150 cm	Take Off Launches drone to automated height. TRY Scheduled launch sequences with controlled delay intervals, autonomous mission initiation, or trigger-based flight launches.
land 105 speed	Land Triggers a stable descent. TRY Emergency landing procedures on low battery, auto-landing at mission completion, or failsafe descent on signal loss.
arm	Arm Enables motor power.
flip	Flip Performs an automatic 360° flip in the chosen direction. Needs height, clear space all round and a healthy battery. This is not an emergency stop: that is <i>disarm</i> , below. TRY Trigger a flip from an RC switch, but only once armed and well clear of people and ceilings.
disarm	Disarm Cuts power to all motors immediately, whatever the drone is doing. Unlike land , there is no controlled descent: the drone stops flying at once and will drop if airborne. This is the emergency stop, not a normal way to end a flight. TRY Wire it to a crash or low-battery condition as an emergency stop.

PROPELLERS ON, READ THIS FIRST

These are the only blocks that make the drone fly by itself. Use them in a netted area or wide open space, with a pilot on the controls ready to take over, and never for the first test of new logic.

A supervised flight pattern

STEP	BLOCKS
1 · Gate the take-off	<i>If Ok to Arm</i> and an RC stick is pushed → <i>Take Off</i>
2 · Hover	Pilot flies manually; your program only watches
3 · Land on trouble	<i>If Is Signal Lost</i> or <i>Is Low Battery</i> → <i>Land</i>
4 · Cut on crash	<i>If Is Crashed</i> → <i>Disarm</i>

Land or disarm?

BLOCK	WHAT HAPPENS
land	A controlled descent. The flight controller brings the drone down at the speed you set, then stops the motors. This is the normal way to end a flight.
disarm	Motors cut immediately, with no descent. If the drone is airborne it will drop. This is an emergency stop, not a landing.

WIRE DISARM TO SOMETHING YOU TRUST

Because it drops the drone, only trigger *disarm* from a condition you have already tested, such as *Is Crashed* or a dedicated switch. Prefer *land* for anything routine.

CHAPTER SIXTEEN

Motor Drive

Direct speed control of individual motors, for ground robots, gimbals, winches and bench tests.

Drive the motors directly, bypassing the flight mixer. Intended for bench testing, ground robots and mechanisms rather than flight.

BLOCK	WHAT IT DOES
	Motor M1 Reversible motor (1000-2000 PWM). <i>TRY Autonomous ground rover locomotion with precise bi-directional movement control or mechanical landing gear actuation during flight sequences.</i>
	Motor M2 Reversible motor (1000-2000 PWM). <i>TRY Automated cargo bay door sequencing, retractable payload mechanisms, or remote-controlled mechanical actuator management.</i>
	Motor M5 Propeller thrust (1000-2000 PWM). <i>TRY With propellers removed, spin one motor at a low PWM and confirm the right arm responds.</i>
	Motor M6 Propeller thrust (1000-2000 PWM). <i>TRY With propellers removed, compare how the four arms behave at the same PWM value.</i>
	Motor M7 Propeller thrust (1000-2000 PWM). <i>TRY Individual propeller performance testing, thrust balancing verification, or asymmetric flight pattern generation.</i>
	Motor M8 Propeller thrust (1000-2000 PWM). <i>TRY Motor response curve tuning, connectivity verification, or differential thrust maneuvers for advanced flight control.</i>

PROPELLERS OFF. EVERY TIME.

These blocks bypass the flight mixer and stabilisation. A wrong number spins a propeller at full speed with nothing to catch it. Remove all propellers before running anything on this page.

PWM VALUE	EFFECT	USE
1000	Stopped	Idle and safe state, end here
1100-1300	Slow	Bench testing, ground robots
1400-1700	Medium	Normal drive
1800-2000	Fast to full	Handle with care

FINISH AT 1000

Put a block in *on stop* that sets every motor you used back to 1000. Stopping the program should always stop the motors.

CHAPTER SEVENTEEN

Peripheral Control

Connect external hardware: buttons, switches, servos, LEDs and analog sensors.

Connect the outside world to the drone: read buttons and analog sensors, drive digital outputs, and control servos or other PWM devices.

BLOCK	WHAT IT DOES
	Read ADC Reads analog voltages (0–3.3V) as raw digital numbers. <i>TRY Distance sensor integration for obstacle avoidance, light intensity monitoring, environmental condition measurement, or custom sensor payload data acquisition.</i>
	Digital Write Sets a digital pin to HIGH (1) or LOW (0). <i>TRY External relay control for high-power devices, warning beacon activation, landing gear deployment, or payload mechanism triggering.</i>
	Servo Control Controls the angle of standard PWM servos. <i>TRY Payload delivery systems, mechanical gripper control, camera gimbal positioning, autonomous object manipulation, or precise mechanical actuation tasks.</i>
	Digital Read Reads the binary state (High/Low) of an input pin. <i>TRY Limit switches for mechanical arms, user input buttons, or simple on/off sensor integration.</i>
	PWM Output Generates a Pulse Width Modulation signal on a pin. <i>TRY Variable LED brightness control, DC motor speed regulation, or generating analog-like signals.</i>

3.3 V LOGIC

The flight controller's pins are not 5 V tolerant. Use a level shifter for 5 V sensors, and power servos from their own supply, sharing only ground, or the drone may brown out and reset mid-flight.

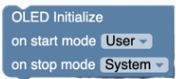
DETECT THE PRESS, NOT THE HOLD

The loop repeats every LoopTime, so "button is down" stays true for many passes in a row. Store the previous state in a Boolean variable and act only when it changes, which registers one press.

CHAPTER EIGHTEEN

OLED Display

A small screen on the drone. Set it up, clear it, draw, then push the frame. That cycle is all there is to it.

BLOCK	WHAT IT DOES
	OLED Initialize Boots up the physical monochromatic OLED display screen. <i>TRY Place in your "on start" block to trigger screen boot sequence before flight.</i>
	OLED Clear Screen Clears all pixels from the display buffer, turning the screen black. <i>TRY Place at the very beginning of your "pluto loop" to wipe the screen clean before drawing the next frame.</i>
	OLED Update Frame Pushes all buffered text, values, and shape drawings onto the physical screen. <i>TRY Place at the absolute end of your "pluto loop" after all rendering blocks are defined.</i>
	OLED Draw Text Draws a text string at specified (X, Y) coordinates (0,0 is top-left). <i>TRY Displaying section titles like "ALTITUDE", "BATTERY STATUS", mode names, or "GO!!!".</i>
	OLED Draw Number Renders a numerical value or active variable at specified (X, Y) coordinates. <i>TRY Displaying live drone height sensor readings, battery cell voltage percentages, or scoreboard numbers.</i>
	OLED Draw Pixel Turns a single pixel on or off at the specified (X, Y) coordinate. <i>TRY Plotting complex customized path graphs or fine-grain visual indicators.</i>
	OLED Draw Line Draws a straight line from coordinate (X0, Y0) to (X1, Y1). <i>TRY Creating a center court dividing line for Pong arcade, or rendering scale axes for Z-altitude radar.</i>
	OLED Draw Rect Draws a solid filled rectangle of width W and height H starting at (X, Y). <i>TRY Drawing visual progress bars, player paddles, or filled warning notification window shapes.</i>

THE DRAW CYCLE

Every frame follows the same three steps: **Clear Screen** → draw everything → **Update Frame**. Miss the update and nothing appears; miss the clear and the old frame smears into the new one.

COORDINATES

The screen is 128 × 64 pixels with **(0,0) at the top-left**. X runs right and Y runs down, so a larger Y is lower on the screen. Continued overleaf.

OLED Display continued

Shapes, pixels and lines, the drawing blocks.

BLOCK	WHAT IT DOES
	OLED Rect Outline Draws a hollow wireframe rectangle of width W and height H starting at (X, Y). <i>TRY Creating border frames for tables, grouping different data elements, or rendering outline boxes for game UI.</i>
	OLED Draw Circle Draws a hollow circle with center coordinates (cx, cy) and radius r. <i>TRY Rendering telemetry radar screens, target scopes, or circular dials for drone orientation.</i>
	OLED Erase Area Erases all pixels in a rectangular area of width W and height H starting at (X, Y) by turning them off. <i>TRY Selectively clearing dynamic number display regions before rendering new readings without flashing the entire screen.</i>
	OLED Round Eyes Draws robotic round eyes on the display with customizable pupil offsets (pupilX, pupilY). <i>TRY Building an interactive robot face for the drone that rolls or pitches its pupils to indicate movement direction.</i>
	OLED Dead Eyes Draws crossed dead eyes on the display, indicating a critical event. <i>TRY Triggering a skull/dead-eyes screen mode when the drone crashes, hits low battery, or is disarmed.</i>
	OLED Draw Joysticks Renders a visual mapping of the RC joystick controls (throttle, yaw, roll, pitch values) as coordinate crosshairs. <i>TRY Displaying live pilot stick positions on the screen for telemetry monitoring and pilot flight analysis.</i>
	OLED Pitch Line Draws a horizon-style tilt/pitch indicator line corresponding to the drone's current pitch angle in degrees. <i>TRY Building an artificial horizon HUD dashboard to visually see the drone's real-time pitch attitude.</i>
	OLED Roll Line Draws a horizon-style tilt/roll indicator line corresponding to the drone's current roll angle in degrees. <i>TRY Building an artificial horizon HUD dashboard to visually see the drone's real-time roll attitude.</i>

A live telemetry screen

ORDER	BLOCK
In on start	OLED Initialize
Top of <i>pluto</i> loop	OLED Clear Screen
Then	Draw Text "ALT" · Draw Number Current Height
Then	Draw Text "BAT" · Draw Number Battery Status
End of <i>pluto</i> loop	OLED Update Frame

ROBOT EYES

Draw two filled circles and move them with Current Roll and Pitch. The drone then appears to look around as you tilt it.

CHAPTER NINETEEN

Ranging Sensors

Laser rangefinders that measure distance in four directions, and can stop the drone hitting things on their own.

Laser rangefinders measure distance in four directions. Read them yourself, or let the firmware steer away from obstacles automatically.

BLOCK	WHAT IT DOES
	XRanging Init All Initializes all range sensors (Left, Right, Front, Back) at once. <i>TRY</i> Booting up the entire collision-guard array in "on start" before takeoff.
	XRanging Init Sensor Initializes a specific directional sensor (Left, Right, Front, Back) with a distance threshold. <i>TRY</i> Setting up a single directional safety fence (e.g. forward laser sensor) at a specific detection threshold.
	XRanging Init OA Configures the automatic object avoidance algorithm with an avoidance distance. <i>TRY</i> Setting up a default 30 cm avoidance bubble before executing flight paths.
	XRanging Enable OA Enables the automatic obstacle avoidance system overrides. <i>TRY</i> Turn on obstacle avoidance so the drone automatically backs away when close to walls or furniture.
	XRanging Disable OA Disables the automatic obstacle avoidance overrides. <i>TRY</i> Temporarily disable automatic corrections to execute tight-fit indoor landing or precise positioning.
	XRanging Triggered? Checks if a specific directional range sensor is triggered (object closer than threshold). <i>TRY</i> Triggering warning beep sounds, flashing the red LED, or printing "WARNING" on the OLED display.
	XRanging Get Range Retrieves the actual measured distance in centimeters (cm) from a specific sensor. <i>TRY</i> Building a dynamic distance display on the OLED, or writing proportional navigation code to track a target.

Two ways to use them

APPROACH	HOW IT WORKS
Automatic. Init OA, then Enable OA	The firmware backs the drone away from obstacles for you. Nothing to program.
Manual, read the distance yourself	You decide what happens: stop, turn, light an LED, print a warning.

INITIALISE BEFORE YOU READ

A sensor that was never initialised returns no useful distance. Put your Init blocks in *on start*, before anything tries to read them.

PROXIMITY ALARM

Read the front sensor, and light the red LED plus print a warning when the distance drops below 30 cm. Wheel the drone towards a wall by hand and watch it react, no flying needed.

APPENDIX A

Project Workbook

One project per category for 18 of the 20, ordered so each builds on the last. Fundamentals first, using things you can see without flying, then the drone itself, then timers and functions.

HOW TO READ A ROW

The description gives the concept plus a worked **example**, build that first. The **challenge** is a variation to try on your own once the example works.

SAFETY FOR PROJECTS 13–16

Keep a hand on the controls, test motors with the **propellers removed**, and always have a way to disarm.

Part 1 · Programming you can see

#	PROJECT & BLOCKS	BUILD THIS, THEN TRY THE CHALLENGE
1	Build Your First Program CORE BLOCKS Select Receiver Mode, On Start, Pluto Loop, On Stop	Understand the structure of every program, initialise once, run continuously, stop safely. Example: turn the Red LED ON in <i>On Start</i> and OFF in <i>On Stop</i> to see when each block runs. CHALLENGE Add a Pluto Loop that keeps the Blue LED blinking between start and stop.
2	Create LED Patterns LED CONTROL Red LED, Green LED, Blue LED	Control the built-in LEDs for visual feedback and status. Example: toggle the Red LED ON and OFF in the loop. CHALLENGE Blink the Green LED instead, then build a Red → Green → Blue traffic-light sequence.
3	Monitor Your Program CONSOLE Print Message, Print Text + Value	Debug and watch your program live in the Pluto Monitor. Example: print "Program Started", then continuously print a counter with <i>Print Text + Value</i> . CHALLENGE Print a live sensor value (e.g. battery %) instead of the counter.
4	Build a Counter VARIABLES Create Integer, Set Variable, Change Variable	Variables store information that changes while the program runs. Example: create an integer and add 1 each loop to count how many times it ran. CHALLENGE Reset the counter to 0 each time it reaches 50.
5	Calculate and Display MATH Arithmetic, Round, Clamp, Modulo	Perform calculations and shape values for control and decisions. Example: calculate the average of two numbers and <i>Round</i> it before printing. CHALLENGE Use <i>Modulo</i> on a counter to cycle through three LED states; <i>Clamp</i> a value to 1000–2000.
6	Repeat Without Copying LOOPS Repeat, For Loop, Break	Run instructions many times without duplicating blocks. Example: use <i>Repeat</i> to blink an LED exactly five times at startup, then stop with <i>Break</i> . CHALLENGE Use a <i>For Loop</i> counter to blink each LED colour in turn.

Project Workbook 7–12

Now the drone itself: sensors, state, safety, the pilot's sticks and external hardware.

Part 2 · Reading the drone

#	PROJECT & BLOCKS	BUILD THIS, THEN TRY THE CHALLENGE
7	Smart Decisions LOGIC <i>If-Then, Compare, AND, NOT, Boolean</i>	Make decisions from conditions and combine several checks. Example: turn the Green LED ON only when the counter is greater than 20. CHALLENGE Extend it: Green ON only when counter > 20 AND battery is NOT low.
8	Live Sensor Dashboard SENSOR DATA <i>Accelerometer, Magnetometer, Barometer, Battery Status</i>	Read real-time sensor values from the drone. Example: read the Accelerometer and print its value live in the Pluto Monitor. CHALLENGE Add Magnetometer, Barometer and Battery readings; light an LED when acceleration exceeds a threshold.
9	Understand Drone Position CURRENT STATE <i>Current Roll, Pitch, Height, App Heading</i>	Access the drone's live orientation and altitude. Example: read Current Height and print it continuously. CHALLENGE Read Roll/Pitch too and turn on the Blue LED when height is above 30 cm.
10	Build a Safety System FLIGHT STATUS <i>Ok to Arm, Is Armed, Is Low Battery, Is Signal Lost, Is Crashed</i>	Monitor the drone's health and safety state before and during flight. Example: light the Blue LED when <i>Is Armed</i> is true. CHALLENGE Build a full indicator: Green = Ready, Blue = Armed, Red = low battery / signal lost / crash.
11	Read Pilot Inputs REMOTE CONTROL <i>Read RC Channels, Set RC Channels</i>	Read stick positions (1000–2000) and optionally override them with software values. Example: read the Pitch channel and print its value. CHALLENGE Read Throttle or Roll instead, and light an LED when it passes mid-stick.
12	Connect External Hardware PERIPHERAL CONTROL <i>Digital Read, Digital Write, PWM, Servo, ADC</i>	Interface with buttons, sensors, servos and other external electronics. Example: use <i>Digital Read</i> to detect a button press and toggle an LED. CHALLENGE Move a Servo on button press and vary LED brightness with PWM from an analog (ADC) sensor.

STILL ON THE GROUND

Every project on this page works with the propellers off. You are reading and reacting rather than flying, which is the safest way to build confidence in your logic before it controls a motor.

Project Workbook 13–18

Guided flight, motors and flight commands, then timers and functions applied to what you have already built.

Part 3 · Flying, and power-ups

#	PROJECT & BLOCKS	BUILD THIS, THEN TRY THE CHALLENGE
13	Guided Flight DESIRED STATE <i>Desired Roll, Desired Pitch, Desired Yaw</i>	Command target angles while the flight controller keeps the drone stable. Example: set a small Desired Roll for one second to make the drone lean gently. CHALLENGE Return to level automatically, then try a gentle Desired Pitch nudge.
14	Mode Indicator FLIGHT MODE <i>Headfree, Throttle</i>	Detect the active flight mode and show it visually. Example: turn on the Green LED when Headfree mode is active. CHALLENGE Add Throttle Mode and show a unique LED colour for each mode.
15	Motor Bench Test MOTOR DRIVE <i>Motor M1, M2, M5–M8</i>	PWM values (1000–2000) drive the motors, test safely on the bench. Example: with props removed, set Motor M1 to a low PWM and print the value. CHALLENGE Gradually ramp the motor up and down with a counter; repeat for M5/M6.
16	Supervised Flight Sequence FLIGHT COMMANDS <i>Take Off, Land, Flip, Disarm</i>	Combine high-level flight commands with safety logic, with the pilot supervising on the RC. Example: trigger Take Off from an RC stick, hold a hover under manual control, then Land. CHALLENGE Auto-land if the link is lost and Disarm on crash or low battery; add a Flip only when armed and safely airborne.
17	Time-Based Events TIMER <i>Set Timer, Check Timer, Reset Timer</i>	Act after precise time intervals instead of every loop cycle. Example: set a 2-second timer and change the LED colour each time it finishes, then Reset. CHALLENGE Build a stopwatch that prints elapsed seconds and flashes an LED every 500 ms.
18	Reuse Your Code CREATE FUNCTION <i>Define Function, Call Function</i>	Group blocks into named, reusable functions to keep programs clean. Example: create a Warning() function that blinks the Red LED and prints a message, then Call it. CHALLENGE Call Warning() from two different conditions (low battery and crash) instead of repeating blocks.

PROJECTS 15 AND 16 INVOLVE MOTORS

Run 15 with the propellers removed. For 16, fly in a netted area or wide open space with a pilot on the controls ready to take over at any moment, and build the auto-land and disarm safety logic *before* the take-off logic.

GOING FURTHER

These projects cover 18 of the 20 categories; the OLED Display and Ranging Sensors chapters are yours to explore. To go further, combine them: an OLED dashboard driven by ranging sensors, a rover built on Motor Drive and Logic, or a light display that reacts to attitude. Beyond that, the Pluto IDE API Handbook covers the same drone in C++.

APPENDIX B

Safety & Troubleshooting

Read this page before the first flight of every session.

Pre-flight checklist

BEFORE POWER-ON

- Propellers the right way round and tight
- No cracked propellers or loose arms
- Battery charged and firmly seated
- Drone level on a flat surface
- Everyone at least 2 m clear

BEFORE DEVELOPER MODE

- You know what your program will do
- *Ok to Arm* checked in your logic
- Disarm condition built and tested
- Pilot ready on the controls
- New logic tested with propellers off

THE MOST IMPORTANT RULE

Run new or modified blocks with the propellers **removed** first. Confirm the behaviour on the LEDs and the console, then refit propellers and fly in a netted area or clear open space.

Classroom setup

PRACTICE	WHY
One supervising adult per flying drone	Someone must always be able to cut power.
Fly inside a net or safety cage	Contains fly-aways from programming mistakes.
Eye protection within 2 m	Propeller fragments travel fast.
Charge batteries in a fire-safe bag, never unattended	Damaged cells can vent and ignite.
Retire any battery that is puffed, hot or dented	Swelling means internal damage.

Troubleshooting

SYMPTOM	LIKELY CAUSE AND FIX
Program does nothing after upload	Developer Mode is off. Blocks only run while it is enabled.
Counter never counts up	The variable is being created inside <i>pluto loop</i> . Create it in <i>on start</i> .
LED blinks too fast to see	The loop runs very fast. Gate it with a <i>Timer</i> .
Button fires many times	You are testing the level, not the change. Store the previous state and act on the edge.
Nothing appears on the OLED	Missing <i>Update Frame</i> at the end of the loop, or <i>Initialize</i> in <i>on start</i> .
Ranging block returns nothing useful	Sensor never initialised. Add the Init block to <i>on start</i> .
Drone will not arm	Print <i>Ok to Arm</i> . Usually not level, still calibrating, or low battery.
Drone drifts or leans by itself	A Desired State block was left set. Return desired angles to 0 when a manoeuvre ends.
Flight becomes jerky after adding blocks	Something is holding up the loop. Replace long <i>Repeat</i> loops with a <i>Timer</i> .
Servo jitters or the board resets	Servo drawing power from the 3.3 V rail. Give it its own supply; share only ground.
Heading is wrong near a desk	Magnetometer interference from steel furniture or speakers. Recalibrate in the open.

STILL STUCK?

The interactive guide at github.com/DronaAviation/Plutoblocks-guide-book shows a picture of every block described in this handbook.

APPENDIX C

Block Index

All 109 blocks in PlutoBlocks, A–Z, with the page documenting each. Names are exactly as they appear in the app.

BLOCK	CATEGORY	PG
Accelerometer	Sensor Data	14
Arithmetic	Math	10
Arm	Flight Command	19
Atan2	Math	10
Barometer	Sensor Data	14
Battery Status	Sensor Data	14
Blue LED	LED Control	8
Boolean	Logic	11
Both True? (AND)	Logic	11
Break/Continue	Loops	12
Call Function Code	Create Function	13
Check Timer	Timer	12
Clamp (Limit)	Math	10
Compare	Logic	11
Constant	Math	10
Create Boolean	Variables	9
Create Float	Variables	9
Create Integer	Variables	9
Current Height	Current State	15
Current Pitch	Current State	15
Current Roll	Current State	15
Current Yaw	Current State	15
Custom C++ Code	Create Function	13
Define Function Code	Create Function	13
Desired Pitch	Desired State	18
Desired Roll	Desired State	18
Desired Yaw	Desired State	18
Digital Read	Peripheral Control	21

BLOCK	CATEGORY	PG
Digital Write	Peripheral Control	21
Disarm	Flight Command	19
Flip	Flight Command	19
For Loop (Counter)	Loops	12
Get App Heading	Current State	15
Green LED	LED Control	8
Gyroscope	Sensor Data	14
Headfree Mode	Flight Mode	16
If This, Then That	Logic	11
Is Armed	Flight Status	16
Is Crashed	Flight Status	16
Is Inflight Low Battery	Flight Status	16
Is Low Battery	Flight Status	16
Is Signal Lost	Flight Status	16
Land	Flight Command	19
Magnetometer	Sensor Data	14
Modulo	Math	10
Motor M1	Motor Drive	20
Motor M2	Motor Drive	20
Motor M5	Motor Drive	20
Motor M6	Motor Drive	20
Motor M7	Motor Drive	20
Motor M8	Motor Drive	20
Not	Logic	11
Null	Logic	11
Number	Math	10
Number Property	Math	10
Ok to Arm	Flight Status	16

Block Index continued

BLOCK	CATEGORY	PG
OLED Clear Screen	OLED Display	22
OLED Dead Eyes	OLED Display	23
OLED Draw Circle	OLED Display	23
OLED Draw Joysticks	OLED Display	23
OLED Draw Line	OLED Display	22
OLED Draw Number	OLED Display	22
OLED Draw Pixel	OLED Display	22
OLED Draw Rect	OLED Display	22
OLED Draw Text	OLED Display	22
OLED Erase Area	OLED Display	23
OLED Initialize	OLED Display	22
OLED Pitch Line	OLED Display	23
OLED Rect Outline	OLED Display	23
OLED Roll Line	OLED Display	23
OLED Round Eyes	OLED Display	23
OLED Update Frame	OLED Display	22
on start	Core Blocks	6
on stop	Core Blocks	6
pluto loop	Core Blocks	6
Print Message	Console	8
Print Text + Value	Console	8
PWM Output	Peripheral Control	21
Random Fraction	Math	10
Random Integer	Math	10
Read ADC	Peripheral Control	21
Read RC Pitch	Remote Control	17
Read RC Roll	Remote Control	17
Read RC Throttle	Remote Control	17

BLOCK	CATEGORY	PG
Read RC Yaw	Remote Control	17
Red LED	LED Control	8
Repeat	Loops	12
Repeat While	Loops	12
Reset Timer	Timer	12
Round	Math	10
Select Receiver Mode	Core Blocks	6
Servo Control	Peripheral Control	21
Set RC Pitch	Remote Control	17
Set RC Roll	Remote Control	17
Set RC Throttle	Remote Control	17
Set RC Yaw	Remote Control	17
Set Timer	Timer	12
Single Operation	Math	10
Take Off	Flight Command	19
Ternary	Logic	11
Throttle Mode	Flight Mode	16
Trigonometry	Math	10
XRanging Disable OA	Ranging Sensors	24
XRanging Enable OA	Ranging Sensors	24
XRanging Get Range	Ranging Sensors	24
XRanging Init All	Ranging Sensors	24
XRanging Init OA	Ranging Sensors	24
XRanging Init Sensor	Ranging Sensors	24
XRanging Triggered?	Ranging Sensors	24

LOOKING FOR SOMETHING THAT IS NOT HERE?

PlutoBlocks covers the blocks on this list. Anything beyond it, custom PID tuning, direct register access, new drivers, is reachable from C++ via the Pluto IDE API, or by embedding custom C++ inside a Create Function block (page 13).

Notes

Room for your own block combinations, pin assignments and project ideas.



Block programming for a real drone

PlutoBlocks lets a beginner program a drone on the first day, and still has room for advanced work later.

PLUTOBLOCKS

PLUTO X

PLUTO 1.2

MAGIS

INSIDE THIS HANDBOOK

20

Block categories, from LEDs to laser ranging

Programming

LEDs · console · variables · math · logic · loops · timers · functions

109

Blocks documented, each with something to try

The drone

Sensors · state · flight status · RC · desired state · commands · motors

18

Guided projects with challenges, plus a safety checklist

Add-ons

OLED display · ranging sensors · servos, buttons, PWM and ADC

LEARN MORE

dronaaviation.com

github.com/DronaAviation/Plutoblocks-guide-book

Companion volume. Pluto IDE API Handbook

THIS EDITION

PlutoBlocks Guide Book · 2026.2

Software PlutoBlocks

Drones Pluto X · Pluto 1.2

© 2026 Drona Aviation